SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	**Assessment:**	Industry Problem solving task
Weightage:	30%	**Total Marks:**	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	Shen Ivin Clement	**Reg. No.:**	192371023
Section / Batch:	2023-2027	**Date:**	26-08-2026

Instructions to Students

- Identify the relevant concepts of cache hierarchy, SRAM, DRAM, MRAM, NAND Flash, RRAM, memory latency, bandwidth, and virtual memory paging.
- Analyze the memory-access requirements of smartphones, cloud servers, autonomous vehicles, edge AI devices, and gaming consoles, considering latency, bandwidth, capacity, and power consumption.
- Apply suitable memory-hierarchy techniques such as cache optimization, appropriate memory technology selection, data locality, paging optimization, and hierarchical memory organization.
- Compute performance measures such as memory access time, latency, bandwidth, throughput, speedup, hit/miss rates, and resource or power utilization where applicable.
- Interpret the results by evaluating performance improvements, memory bottlenecks, technology trade-offs, and the suitability of the proposed memory hierarchy for each application.

QUESTIONS

1. A mobile device manufacturer observes noticeable lag when users switch rapidly between multiple applications. Analyze how L1, L2, and L3 caches and DRAM participate in storing and retrieving frequently accessed application data. Based on latency, capacity, hit rate, and bandwidth requirements, propose a suitable hierarchical memory organization that can reduce application switching delays and improve overall system throughput.

2. A cloud service provider experiences increased latency when processing database queries from a large number of concurrent users. Analyze how cache hierarchy and virtual memory paging influence memory access performance, and compare their roles in reducing processor-memory latency. Recommend suitable memory technologies and memory-management improvements that can minimize page faults, reduce access delays, and improve database throughput.

3. An autonomous vehicle continuously receives large volumes of sensor data from cameras, LiDAR, radar, and other real-time sensing systems. Evaluate the characteristics of SRAM, DRAM, and MRAM in terms of access latency, bandwidth, capacity, volatility, and power consumption. Based on the real-time processing requirements, design an appropriate memory hierarchy that can provide high data-transfer bandwidth while minimizing access latency and supporting reliable sensor-data processing. 4. An edge-based AI device must load machine-learning models quickly while operating under strict power and storage constraints. Analyze the characteristics of NAND Flash, DRAM, and RRAM with respect to access speed, storage capacity, persistence, power consumption, and suitability for AI workloads. Recommend a hierarchical memory organization that enables fast model loading, efficient intermediate-data processing, reduced energy consumption, and improved overall inference performance.

5. A high-performance gaming console experiences frame drops and noticeable delays when loading large game environments containing high-resolution textures, audio, and other assets. Analyze the interaction between L3 cache and DRAM during the retrieval and processing of frequently accessed game data, considering their latency, capacity, and bandwidth. Propose suitable memory hierarchy enhancements that can reduce data-access bottlenecks, increase memory throughput, and provide smoother frame rendering during large scene transitions.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	**Weightage(%)**	**Level 4 – Exemplary (5 Marks)**	**Level 3 – Proficient (4 Marks)**	**Level 2 – Developing (2–3 Marks)**	**Level 1 – Beginning (0–1 Mark)**
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution

Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Q. No	
Understanding of Industry Problem	**(20)**
Applicati on of Engineeri ng	
Solution Design	**(25)**
Use of Tools	**(15)**
Analysis	**(10)**
Professio na lism	**(5)**
Total (out of 100)	

SIMATS ENGINEERING

Department of Computer Science and Engineering

Assessment Tool 1 — Model Answer Key Industry

Problem Solving Task • CO4 (BL4 — Precision)

Course Code	CSA12	**Course Title**	Computer Architecture

CO Assessed	CO4 — Precision (BL4)	**Weightage**	30% • 100 Marks

CO4 Statement. Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.

Reference — Memory Technology Characteristics (typical values)

SRAM (caches)	~0.5–2 ns	Very high (on-chip)	KB–tens of MB	Volatile — High leakage; 6T cell, low density
DRAM (LPDDR/DDR/HBM)	~50–100 ns	High–very high	GBs	Volatile — Needs refresh; moderate power
MRAM (STT)	~10–40 ns	Moderate	MBs (embedded)	Non-volatile — Low standby power; robust; write slower
NAND Flash	~25–100 µs read	Moderate	GBs–TBs	Non-volatile — Cheap/dense; limited endurance
RRAM / ReRAM	~10–100 ns read	Moderate	MBs (emerging)	Non-volatile — Low read energy; enables compute-in-memory

All figures are representative order-of-magnitude values used consistently across the worked solutions below; exact numbers vary by process node, generation, and vendor.

A mobile device shows lag when users switch rapidly between apps. Analyse how L1, L2, L3 caches and DRAM store and

retrieve frequently accessed application data. Based on latency, capacity, hit rate and bandwidth, propose a hierarchical memory organisation that reduces app-switching delay and improves overall throughput.

1. Understanding the industry problem

On a smartphone, each running app has a **working set** — the code and data it is actively touching. When the user rapidly switches apps, the incoming (foreground) app evicts the previous app's lines from the small on-chip caches. On switching back, the returning app suffers a burst of **cold/compulsory misses** that must be re-fetched from DRAM (or, worse, re-loaded from flash if the app was paged out under memory pressure). Thousands of such misses, each costing ~100 ns, accumulate into the perceptible lag the manufacturer is observing.

2. Role of each level in storing / retrieving app data

• **L1 cache (split I/D, ~32–64 KB, ~1 ns, per-core SRAM):** holds the very hottest instructions and data of the *currently executing* app. Highest speed, smallest capacity — flushed almost entirely on a context switch. • **L2 cache (~256 KB–1 MB, ~4 ns, private/cluster SRAM):** captures the near-term working-set spillover from L1; absorbs short-term reuse within an app.

• **L3 / Last-Level Cache (~4–16 MB, ~15–20 ns, shared SRAM):** the decisive level for app-switching. Being large and shared, it can simultaneously retain lines from *several* recently used apps, so a switch-back can hit in L3 instead of going to DRAM.

• **DRAM (LPDDR5/5X, GBs, ~100 ns, tens of GB/s):** holds the complete state and memory pages of all resident apps. It is the backstop when caches miss, and its **bandwidth** governs how fast a working set can be bulk-reloaded.

3. Performance analysis — Average Memory Access Time (AMAT)

Using the multi-level formula AMAT = $t_{L1} + m_{L1}(t_{L2} + m_{L2}(t_{L3} + m_{L3} \cdot t_{DRAM}))$:

Baseline (small 8 MB L3, poor multi-app retention)

t_{L1}=1 ns, m_{L1}=0.05 · t_{L2}=4 ns, m_{L2}=0.15 · t_{L3}=15 ns, m_{L3}=0.25 · t_{DRAM}=100 ns
AMAT = 1 + 0.05 × (4 + 0.15 × (15 + 0.25 × 100))
= 1 + 0.05 × (4 + 0.15 × 40) = 1 + 0.05 × 10 = **1.50 ns**

Improved (16 MB shared L3 retains background apps → m_{L3} drops 0.25 → 0.10):
AMAT = 1 + 0.05 × (4 + 0.15 × (15 + 0.10 × 100)) = 1 + 0.05 × 7.75 = **1.39 ns**
Average-access speedup = 1.50 / 1.39 = **1.08×**

Why the switch itself improves far more than 8 %

Reloading a 2 MB working set as 64 B lines that all miss to DRAM:
2 MB ÷ 64 B = 32,768 lines × 100 ns = **~3.3 ms** of stall if latency-bound (a visible hitch). If instead served by DRAM bandwidth with prefetch (LPDDR5 ~51 GB/s):
2 MB ÷ 51 GB/s = **~40 µs** — ~80× faster.
If the 2 MB stays resident in a 16 MB shared L3 → reload avoided entirely (µs-class switch).

Interpretation: raw AMAT improves modestly, but the switch-back experience improves by orders of magnitude because a larger shared LLC converts DRAM-latency-bound reloads into cache hits, and prefetch + bandwidth turn any residual reload into a bandwidth-bound (µs) operation rather than a latency-bound (ms) one.

4. Proposed hierarchical memory organisation

• **Keep L1 split & private (~64 KB)** for 1-cycle access to the live app.

• **Private L2 ~512 KB–1 MB per core** to hold each core's near-term working set. • **Enlarge the shared L3/LLC to ~8–16 MB** so the working sets of several recently used apps stay resident — the single biggest lever for switch latency and multi-app hit rate. • **High-bandwidth LPDDR5/5X, multi-channel**, paired with **hardware stride/stream prefetchers** to bulk-reload working sets in the background before the user notices. • **More**

physical RAM (8–12 GB) + zRAM/compressed pages so background apps are not paged out to flash — this removes the slowest (ms-class) reload path entirely. • **Cache partitioning / QoS** so the foreground app cannot completely evict background app lines from the LLC.

5. Result & trade-offs

The redesign targets the two governing parameters — **LLC capacity/hit-rate** and **DRAM bandwidth**. Together they cut switch latency from ms to µs and raise system throughput, giving the smooth switching the manufacturer wants. The trade-off is that larger SRAM caches consume die area and static (leakage) power, and more DRAM raises cost and refresh energy — acceptable given the large responsiveness gain.

A cloud provider sees increased latency processing database queries from many concurrent users. Analyse how cache hierarchy and virtual-memory paging influence memory-access performance and compare their roles in reducing processor-memory latency. Recommend memory technologies and memory-management improvements that minimise page faults, reduce access delays, and improve database throughput.

1. Understanding the industry problem

A database server serves each query by touching buffer-pool pages, index nodes and query structures. Under high concurrency the **combined working set of all sessions can exceed physical RAM**, so the OS begins

demand-paging. Each **page fault** fetches a page from storage (µs–ms), which is 3–5 orders of magnitude slower than an in-memory access. Simultaneously, many processes stress the **TLB**, so more translations miss and trigger page-table walks. The result is rising average latency and long tail latency for queries.

2. Cache hierarchy vs virtual-memory paging — the two levers

Gap it closes	CPU ↔ DRAM latency (ns scale) DRAM ↔ storage capacity gap (µs–ms)

Managed by	Hardware, transparent OS + MMU/TLB, software policy
Unit	Cache line (~64 B) Page (4 KB, or 2 MB/1 GB huge pages)
Miss penalty	L3 miss → DRAM ~100 ns Page fault → SSD ~100 µs (disk ~ms)
Best for	Hot pages, indexes, plan reuse Large address space, isolation, capacity

Comparison of roles: the cache hierarchy trims the *nanosecond* processor-to-memory gap and is invisible to software. Paging manages the far larger *microsecond-to-millisecond* gap between DRAM and storage and provides isolation and a large virtual address space. Under concurrency it is the **paging layer that dominates tail latency**, because a single page fault costs as much as ~1,000 L3 misses.

3. Performance analysis

Effect of page-fault rate on Effective Access Time (EAT)

EAT = $(1 - p)\cdot t_{mem} + p\cdot t_{fault}$, with t_{mem}=100 ns, t_{fault}=100 µs (SSD)=100,000 ns
p = 0.001 (0.1 %): EAT = 0.999×100 + 0.001×100,000 = 99.9 + 100 = **199.9 ns** (≈2× slower!) p = 0.0001 (0.01 %): EAT = 0.9999×100 + 0.0001×100,000 = 99.99 + 10 = **109.99 ns** Speedup from cutting faults 10× = 199.9 / 109.99 = **1.82×**
→ Even a 0.1 % fault rate nearly doubles access time: eliminating faults is the priority.

Effect of TLB hit-rate (huge pages)

EAT = $h\cdot(t_{tlb}+t_{mem}) + (1-h)\cdot(t_{tlb}+t_{walk}+t_{mem})$, t_{tlb}=1, t_{walk}=100, t_{mem}=100 ns
Baseline h=98 %: 0.98×101 + 0.02×201 = 98.98 + 4.02 = **103.0 ns**
Huge pages h=99.8 %: 0.998×101 + 0.002×201 = 100.80 + 0.40 = **101.2 ns**
Huge pages also shrink the page table and page-walk depth — compounding the gain.

4. Recommended memory technologies & management improvements

• **Scale up DRAM (high-capacity DDR5 RDIMMs)** and size the DB **buffer pool** so the hot working set is resident → the page-fault rate *p* collapses toward zero.

• **Fast NVMe SSDs; add a persistent-memory / CXL-attached memory tier** so the residual fault costs µs (not the ms of spinning disk), and capacity can expand elastically under load. • **Enable huge pages (2 MB/1 GB)** to cut TLB misses and page-table overhead.

• **NUMA-aware allocation** so worker threads hit local-node memory and avoid remote latency. • **Better replacement (LRU/CLOCK) + read-ahead prefetch** to keep hot pages and stream scans. • **Connection pooling & admission control** to bound concurrency and prevent thrashing. • **LLC partitioning (e.g., Intel CAT)** to reserve last-level cache for latency-critical queries.

5. Result & trade-offs

Caches remove nanosecond stalls; the throughput breakthrough comes from **eliminating page faults** (more RAM, larger buffer pool, fast NVMe/PMEM) and **reducing TLB pressure** (huge pages), with admission control preventing thrashing. Effective access time falls from ~200 ns toward ~110 ns and tail latency shrinks, raising sustained queries-per-second. Trade-offs: large DRAM and PMEM/CXL add cost and power; huge pages can waste memory through internal fragmentation and must be sized carefully.

An autonomous vehicle continuously receives large volumes of sensor data from cameras, LiDAR and radar. Evaluate SRAM, DRAM and MRAM in terms of access latency, bandwidth, capacity, volatility and power. Based on real-time requirements, design a memory hierarchy that provides high data-transfer bandwidth while minimising access latency and supporting reliable sensor-data processing.

1. Understanding the industry problem

An autonomous driving stack runs a hard-real-time perception loop (typically 10–33 ms per cycle). It must ingest, buffer and process several continuous high-rate sensor streams every cycle, feed them through neural networks, and never miss a deadline — a late frame is a safety hazard. The memory subsystem must therefore deliver **sustained high bandwidth**, **low, predictable latency**, and **reliable, power-cycle-safe** storage of critical state.

2. Evaluation of SRAM, DRAM and MRAM

Access latency	~0.5–2 ns (fastest)	~50–100 ns ~10–40 ns (read < write)
Bandwidth	Very high (on-chip)	High–very high (HBM 400+ GB/s) Moderate
Capacity	KB–tens of MB	GBs MBs (embedded), growing
Volatility	Volatile	Volatile (needs refresh) Non-volatile
Power	Low dynamic, high leakage	Moderate + refresh power Low standby; robust to temp/rad
Best role	Caches / on-chip scratchpad	Frame & feature-map working memory Persistent safety-critical state

3. Performance analysis — bandwidth & buffering

Raw sensor ingest bandwidth

8 cameras × 1920×1080 × 3 B × 30 fps:
1920×1080 = 2,073,600 px × 3 B = 6.22 MB/frame × 30 = 186.6 MB/s per camera × 8 cameras = **~1.49 GB/s**; + LiDAR/radar (~0.2–0.5 GB/s) → **~1.7–2 GB/s aggregate**. Networks re-read feature maps (say 10× reuse): effective demand ≈ 10 × 2 = **~20 GB/s minimum**. → LPDDR5 (~51 GB/s) is marginal for heavy models; HBM2e (~460 GB/s) / GDDR6 (~448 GB/s) give ample headroom.

Deadline & double-buffer checks

Double-buffering one frame from 8 cameras = 2 × 8 × 6.22 MB = **~99.5 MB** → fits DRAM, not SRAM. Per 33 ms cycle the subsystem handles ~2 GB/s × 33 ms = ~66 MB of new data. Moving 66 MB at HBM 460 GB/s takes 66 MB / 460 GB/s = **~0.14 ms** ≪ 33 ms budget. → With HBM, memory bandwidth is NOT the bottleneck (compute is); latency stalls are hidden by buffering.

4. Proposed memory hierarchy

- **On-chip SRAM (L1/L2 + scratchpad) inside the NPU/GPU:** lowest latency and highest on-chip bandwidth for the hottest inference tensors and per-pixel operations.
- **HBM2e/HBM3 (or GDDR6) as accelerator working memory:** hundreds of GB/s to stream frame buffers and CNN feature maps without stalls.
- **LPDDR5 main DRAM for the host CPU:** sensor-fusion buffers, HD maps and general working set.

- **MRAM (non-volatile) for safety-critical persistence:** boot firmware, calibration constants, and a 'last-known-good'/black-box buffer that survives transient power loss — enabling fast, safe restart with no data loss (supports ASIL reliability goals).
- **Ping-pong (double/triple) buffering + ECC:** a new frame is written while the previous is processed, hiding latency and sustaining throughput; ECC on DRAM/MRAM guards data integrity.

5. Result & trade-offs

The hierarchy assigns each duty to the right technology: **SRAM for latency**, **HBM/GDDR DRAM for the multi-GB/s sustained bandwidth**, and **non-volatile MRAM for reliability and persistence** of safety-critical state. Trade-offs: SRAM is tiny though fastest; DRAM is fast and capacious but volatile with refresh power; MRAM is robust and low-standby-power but has higher write latency/energy and lower density than DRAM — hence used for critical state rather than bulk buffers.

An edge-AI device must load ML models quickly under strict power and storage constraints. Analyse NAND Flash, DRAM and RRAM with respect to access speed, capacity, persistence, power and suitability for AI workloads. Recommend a hierarchical organisation enabling fast model loading, efficient intermediate-data processing, reduced energy, and improved inference performance.

1. Understanding the industry problem

An edge-AI device (camera, wearable, IoT node) is battery- and storage-constrained yet must load a model and begin inferencing quickly, often after waking from a low-power sleep. Two pain points dominate: **cold-start latency** (time to get weights from persistent storage into a runnable state) and **energy** (data movement, especially fetching weights, is the largest energy term in many inference workloads — the 'memory wall').

2. Evaluation of NAND Flash, DRAM and RRAM

Access speed	Slow (~25–100 µs page read)	Fast (~50–100 ns) ~10–100 ns read (write slower)

Capacity	High (GBs, cheap)	Moderate (100s MB–few GB) MBs (emerging), dense potential
Persistence	Non-volatile	Volatile Non-volatile
Power	Low idle, moderate active	Moderate + refresh (always-on) Low read energy; ~0 retention
Endurance	Limited P/E cycles	Effectively unlimited Moderate (improving)
AI suitability	Bulk model storage	Activations / intermediates NV weights + compute-in-memory

Key differentiator: RRAM crossbars can perform the matrix-vector multiply *where the weights are stored* (analog **compute-in-memory**), so weights need not be streamed to the compute units each inference — directly attacking both cold-start and energy. Non-volatility also gives **instant-on** with zero standby leakage for the weights.

3. Performance analysis

Cold-start model-load time (100 MB quantised model)

From UFS NAND at 1.5 GB/s: 100 MB / 1.5 GB/s = **~66.7 ms**

From eMMC at 300 MB/s: 100 MB / 300 MB/s = **~333 ms** (clearly perceptible)

Weights resident in non-volatile RRAM (or executed in-place via compute-in-memory): load ≈ **0 ms** → NV weight storage removes the entire 67–333 ms reload on every wake.

Weight-movement energy (why compute-in-memory wins)

Assume ~10 pJ per bit moved from DRAM (illustrative LPDDR value).

One full pass of 100 MB weights = 100×10^{6} B × 8 b × 10 pJ = 8×10^{9} pJ = **8 mJ** per inference. At 30 inferences/s if weights are refetched each time: 30 × 8 mJ = **240 mJ/s ≈ 0.24 W** just for weight traffic.

Keeping weights stationary (RRAM CIM / on-chip SRAM) removes most of this term → multi-× lower inference energy.

4. Recommended hierarchical organisation

• **Tier 0 — compute:** on-chip SRAM buffers for activations, plus optional **RRAM compute-in-memory** tiles that hold weights and perform weight-stationary MAC. • **Tier 1 — working memory:** LPDDR4/5 **DRAM** for activations and intermediate tensors, and for any model too large for on-chip storage.

• **Tier 2 — fast persistent model store: RRAM** (or other fast NVM) holding the weights non-volatile → instant-on, low leakage; the model need not be reloaded from flash on each boot. • **Tier 3 — bulk storage: NAND Flash/UFS** for the model zoo, OTA updates and logs. • **Cross-cutting techniques:** weight **quantisation/pruning** (smaller model → faster load, less DRAM traffic and energy), **operator fusion + tiling** to keep activations on-chip, and **DVFS + duty-cycling** with NV weights for instant wake.

5. Result & trade-offs

NAND provides cheap persistence but slow load; DRAM provides speed and bandwidth but is volatile with always-on refresh power; RRAM provides non-volatile, low-energy storage *and* compute-in-memory that shortens cold-start and slashes weight-movement energy. Storing weights in NVM, running inference weight-stationary, and using DRAM/SRAM only for activations yields fast model loading, lower energy and better inference throughput. Trade-offs: RRAM endurance, write cost and process maturity; DRAM refresh power; NAND's µs-class latency.

A high-performance gaming console suffers frame drops when loading large environments with high-resolution textures, audio and assets. Analyse the interaction between L3 cache and DRAM during retrieval of frequently accessed game data (latency, capacity, bandwidth). Propose memory-hierarchy enhancements that reduce data-access bottlenecks, increase throughput, and provide smoother frame rendering during large scene transitions.

1. Understanding the industry problem

A console must render every frame inside a fixed budget (16.67 ms at 60 fps). A large scene transition floods the pipeline with new high-resolution textures and assets. If those assets are not yet resident, they must stream from the SSD; once in DRAM, the GPU reads them at very high rates. When this new working set overflows the L3 cache, the GPU/CPU become **DRAM-bandwidth bound**, frame time exceeds the budget, and frames are dropped (the observed stutter).

2. Interaction between L3 cache and DRAM

• **L3 / LLC (MBs, ~15–30 ns, high on-chip bandwidth):** captures data reused across frames — draw lists, shader constants, geometry and frequently sampled small textures. Every L3 hit is one less DRAM transaction,
so a high L3 hit-rate directly **reduces DRAM bandwidth demand**.

- **DRAM (unified GDDR6, GBs, ~100 ns, e.g. ~448 GB/s):** holds full-resolution textures, framebuffers and assets. It supplies everything the L3 misses.
- **The coupling:** on a transition, a flood of new textures causes L3 misses → DRAM bandwidth spikes. If demand saturates the bus, and asset-streaming traffic from storage competes for the same bandwidth, the frame's memory traffic no longer fits the per-frame budget → dropped frames.

3. Performance analysis

Per-frame bandwidth budget & streaming cost

60 fps → 16.67 ms/frame. Budget at 448 GB/s = 448 GB/s ÷ 60 = **~7.47 GB of traffic per frame**. A 4K G-buffer (3840×2160 × 4 B = ~33 MB) is read/written many times across shading + post → can approach ~1 GB/frame, competing with texture reads — headroom must be protected. Streaming 2 GB of new textures from a 5.5 GB/s SSD = 2 / 5.5 = **~0.36 s ≈ 22 frames** at 60 fps. → If done as a blocking load it is a visible hitch; it must be spread asynchronously across frames.

How L3 hit-rate frees DRAM bandwidth

DRAM demand = (1 − h) × sampler traffic. Suppose sampler demand = 300 GB/s. h = 0.50 → DRAM demand = 150 GB/s; h = 0.70 → DRAM demand = 90 GB/s.

Raising L3 hit-rate 0.50 → 0.70 cuts DRAM demand 40 % → frees ~60 GB/s for asset streaming.

AMAT for LLC-missing streams = $t_{L3} + m_{L3} \cdot t_{DRAM}$:

m_{L3}=0.50: 15 + 0.50×100 = 65 ns; m_{L3}=0.30: 15 + 0.30×100 = 45 ns → **1.44× faster**.

4. Proposed memory-hierarchy enhancements

- **Large on-die cache ('Infinity Cache'-style) + bigger L3:** raises hit-rate, cutting DRAM bandwidth pressure and creating headroom for streaming.
- **High-bandwidth GDDR6/GDDR6X on a wide bus:** ensure DRAM bandwidth comfortably exceeds worst-case per-frame traffic so transitions do not saturate the bus.
- **Fast NVMe SSD + dedicated hardware decompression + GPU-direct storage path:** assets stream into DRAM at multi-GB/s without a CPU bottleneck (removes the blocking-load hitch). • **Asynchronous / pre-emptive streaming with LOD and mip/virtual texturing:** prefetch upcoming assets before the transition and load only the resolutions and texels actually visible.

- **Texture compression (BCn/ASTC) + cache-friendly tiled/Morton layouts + prefetching:** shrink bandwidth and footprint while raising L3 hit-rate.

5. Result & trade-offs

The bottleneck during transitions is **DRAM-bandwidth saturation plus storage-streaming latency**. Raising the L3 hit-rate lowers DRAM demand, while ample DRAM bandwidth and a fast, hardware-decompressing SSD path let large asset sets stream asynchronously within the per-frame budget — producing stable frame times and smoother rendering. Trade-offs: large on-die caches cost die area and power; higher-bandwidth memory costs power and money; aggressive streaming (LOD, async, virtual texturing) increases engine engineering complexity.